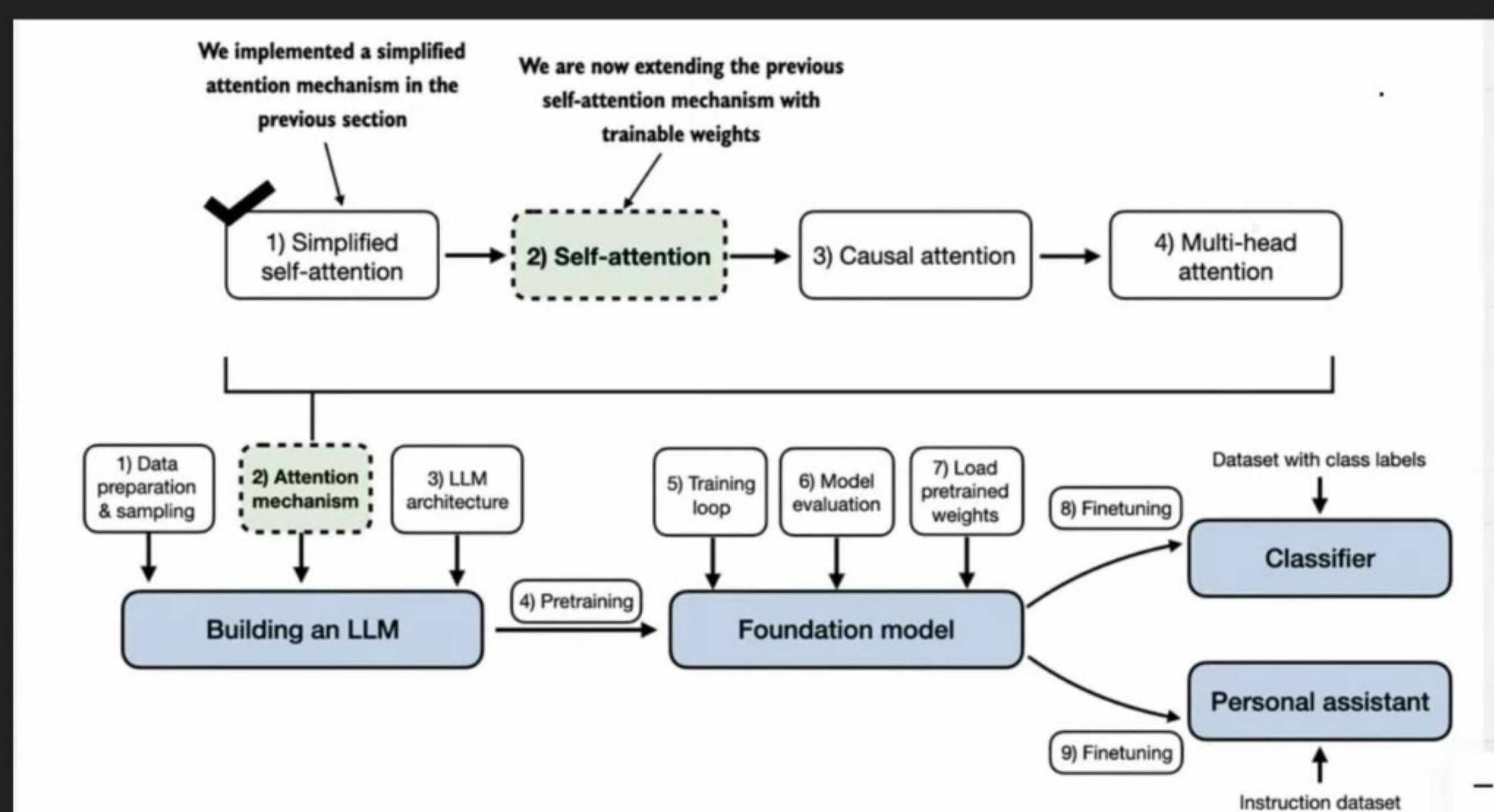


Lecture-15: Self Attention with Trainable weights

In this section, we will learn about the self-attention mechanism used in the original transformer architecture, the GPT models and most other popular LLMs.

This self-attention mechanism is also called as "scaled dot-product attention"



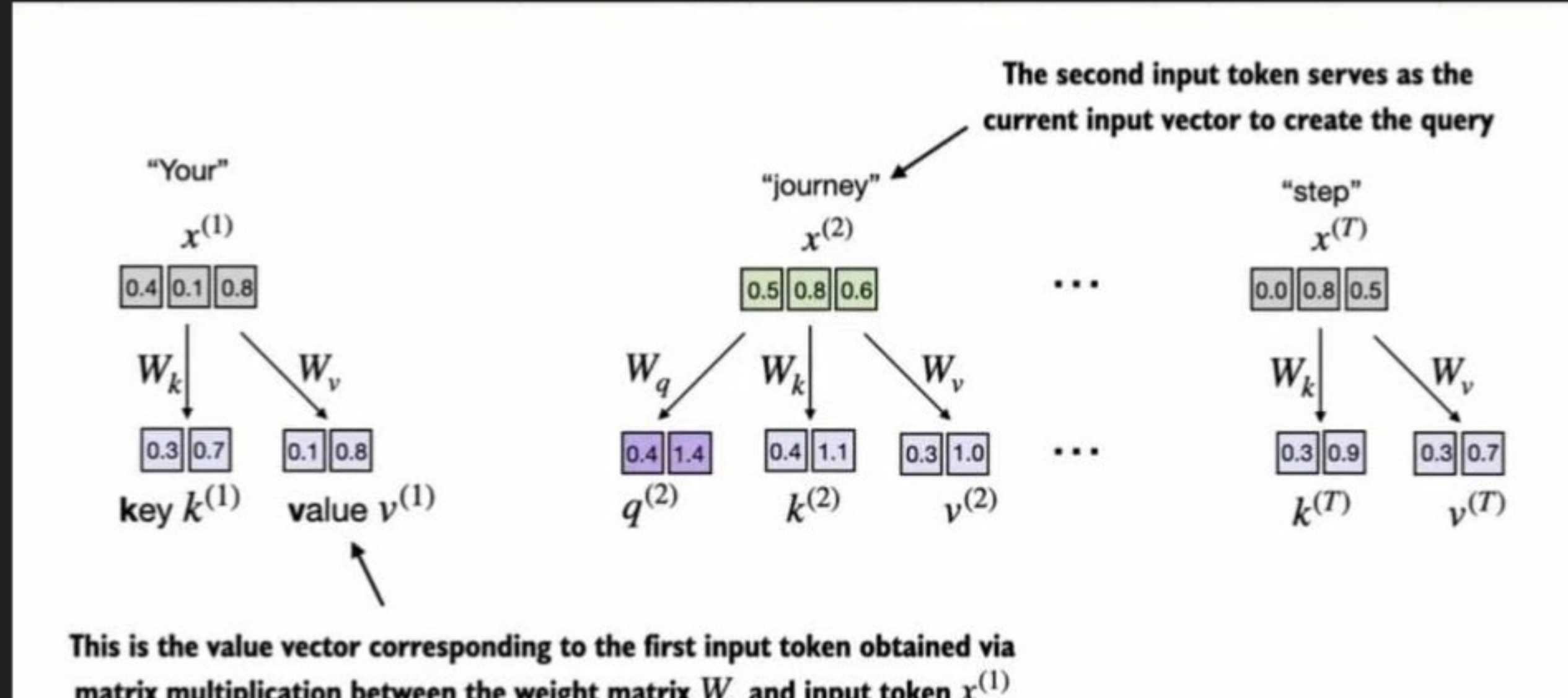
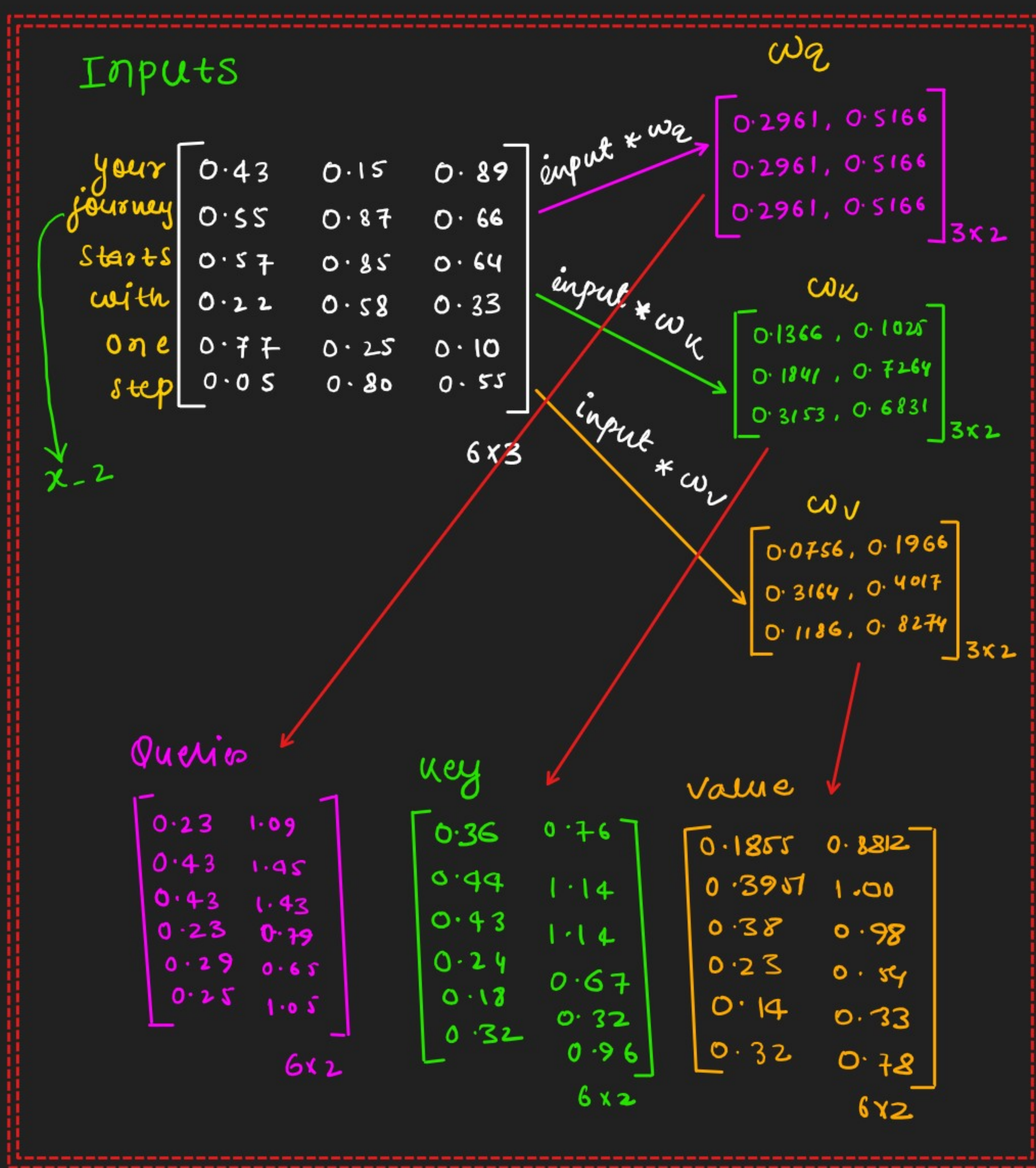
- ① we want to compute context vectors as weighted sum over the input vectors specific to a certain input element
- ② we will introduce weight matrices that are updated during model training
- ③ These trainable weight matrices are crucial so that the model can learn to produce "good context vector"
- ④ we will implement the self-attention mechanism step by step by introducing 3 trainable weight matrices:

w_q
 w_k
 w_v

These 3 matrices are used to project the embedded input $x^{(i)}$ into query, key and value vectors.

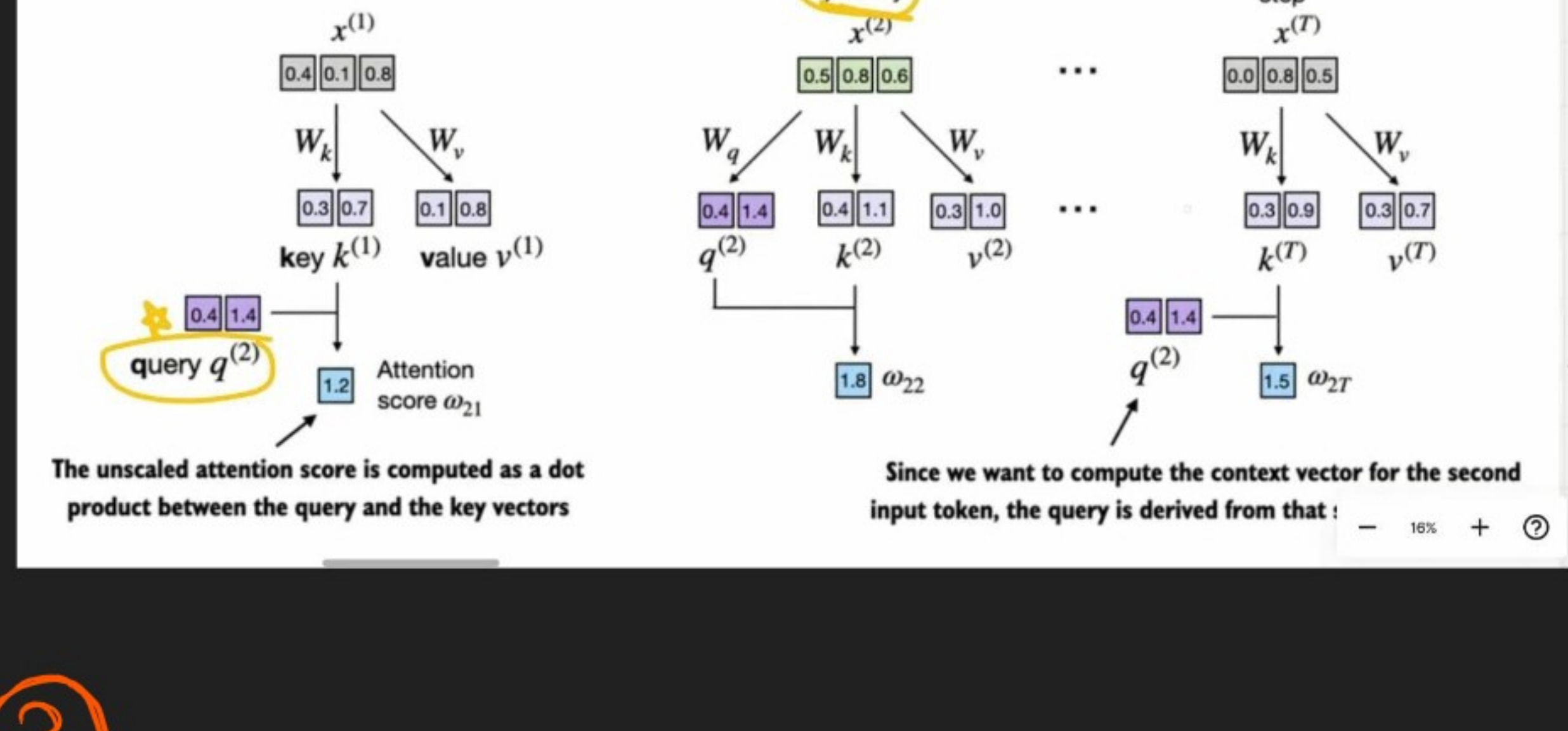
Steps-1: converting input embedding into key, query, value vectors

①

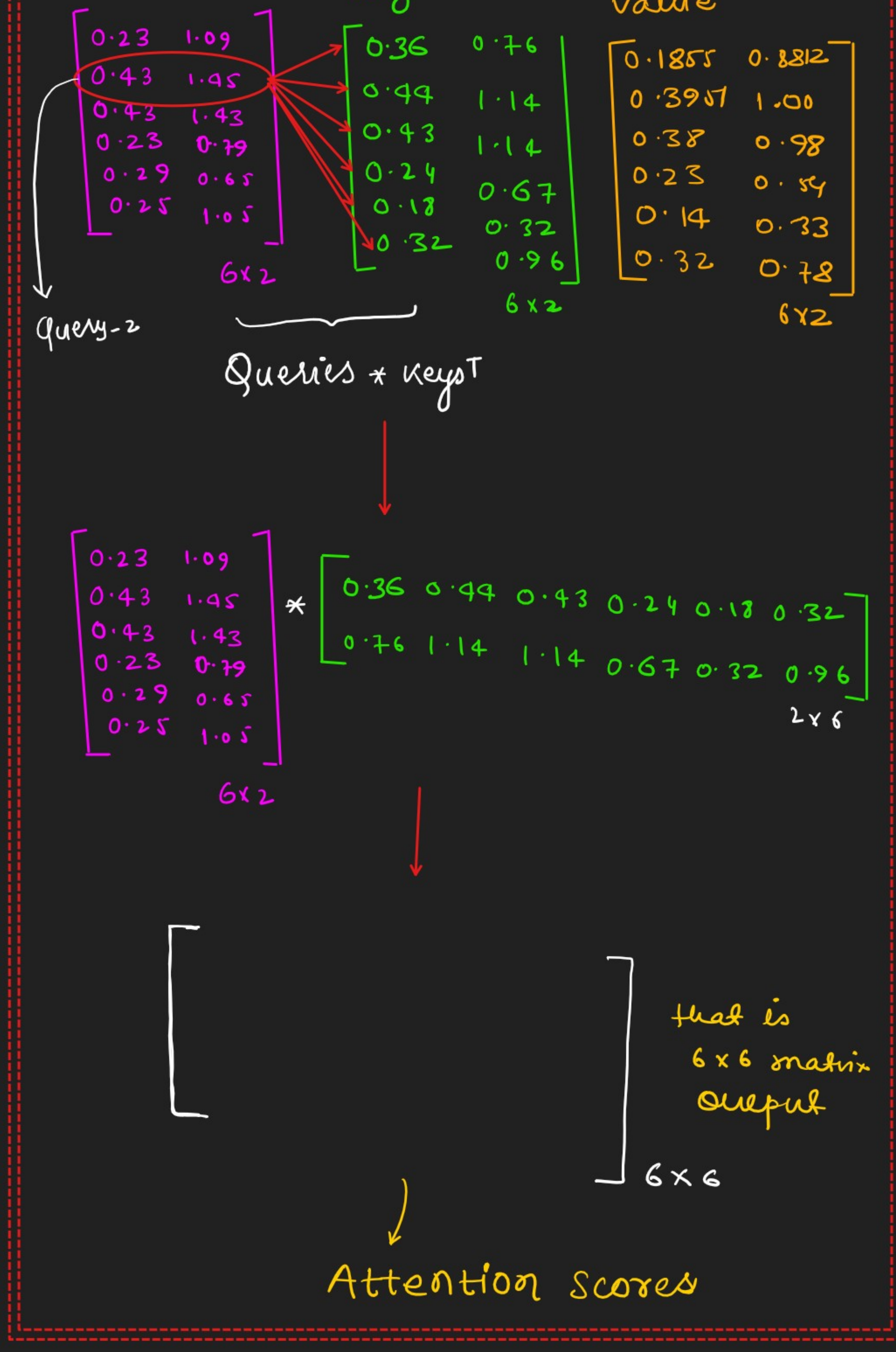


⑤ we will start by computing only one context vector $z^{(2)}$ for illustration purpose

→ computing the attention score (w/o query and key)

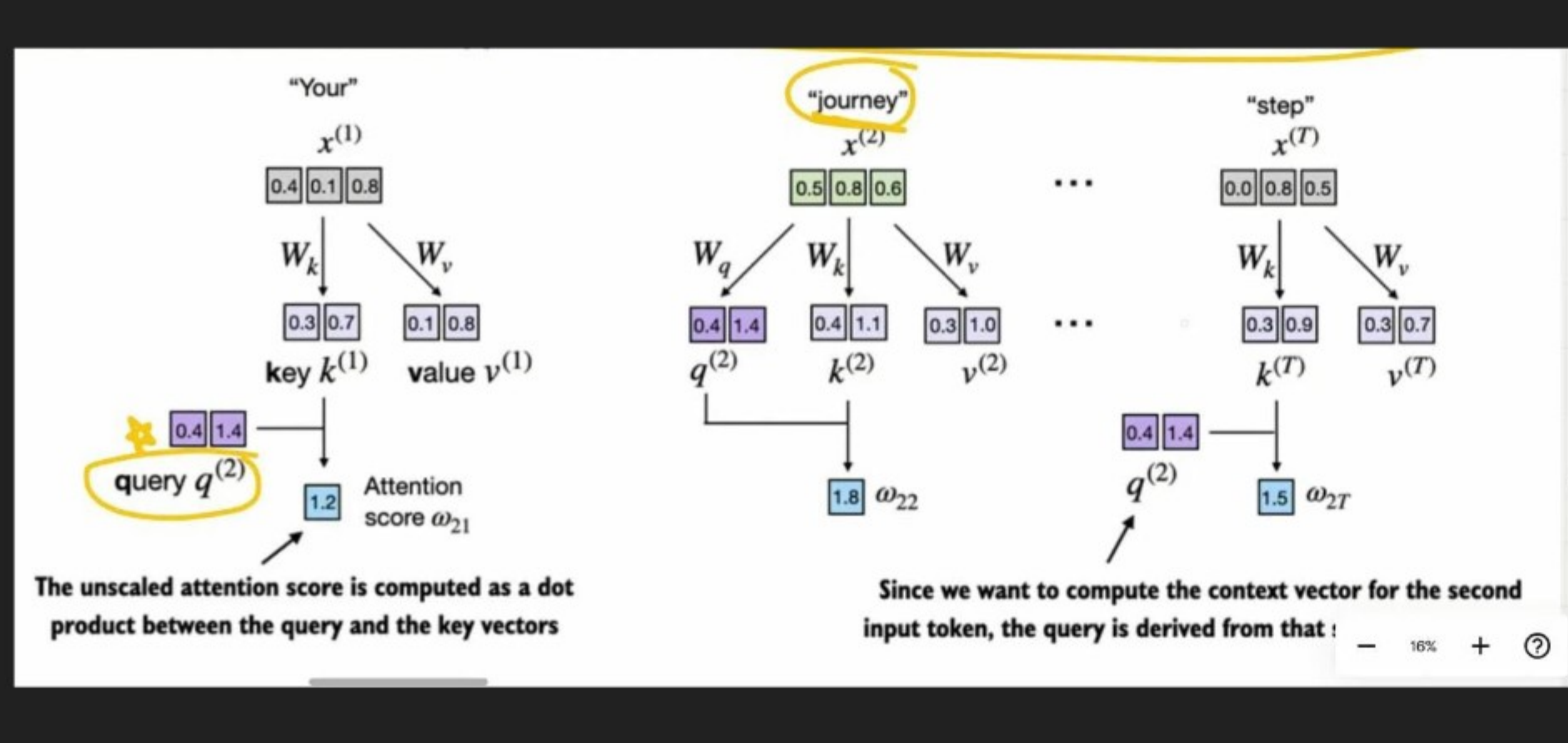
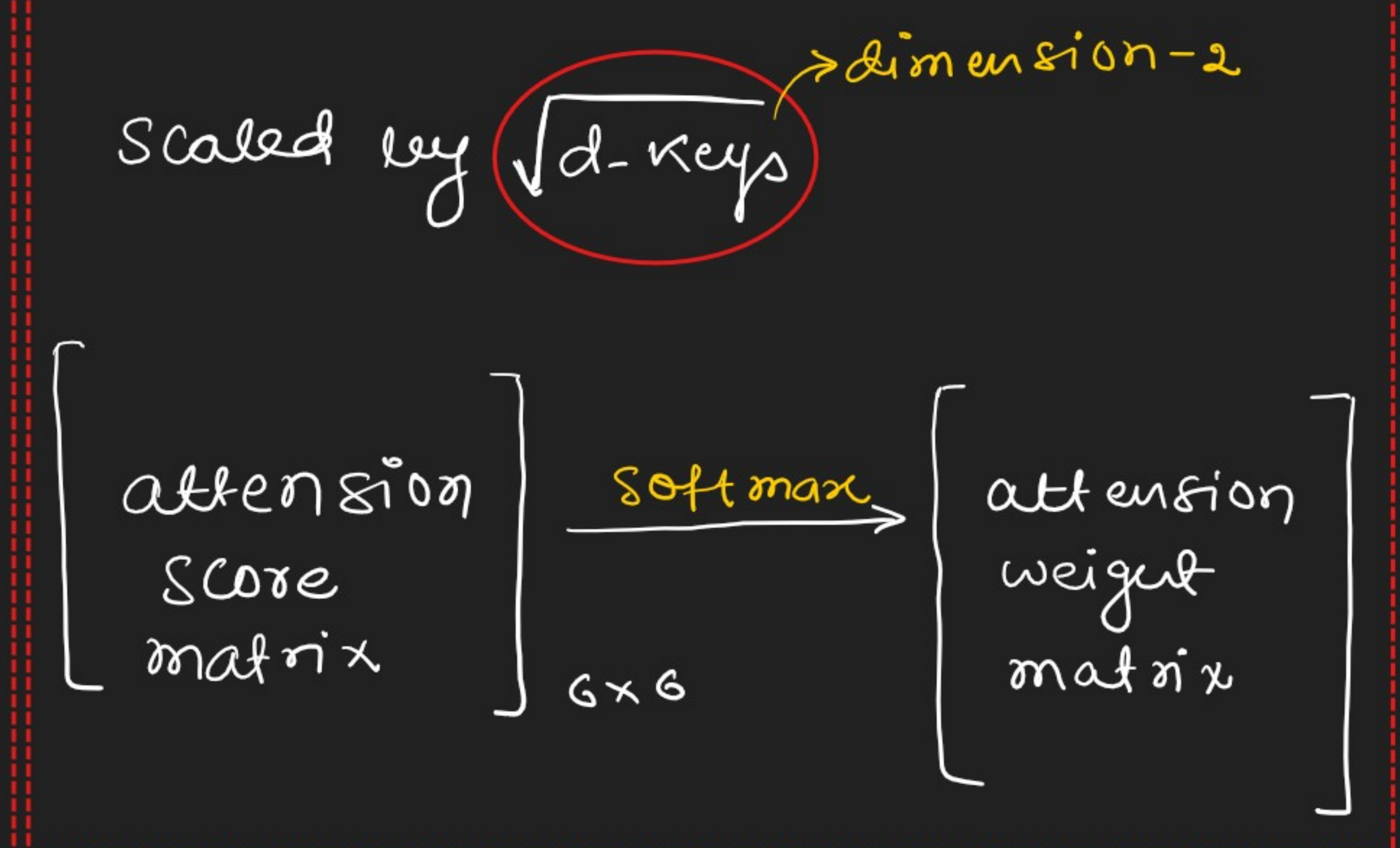


②



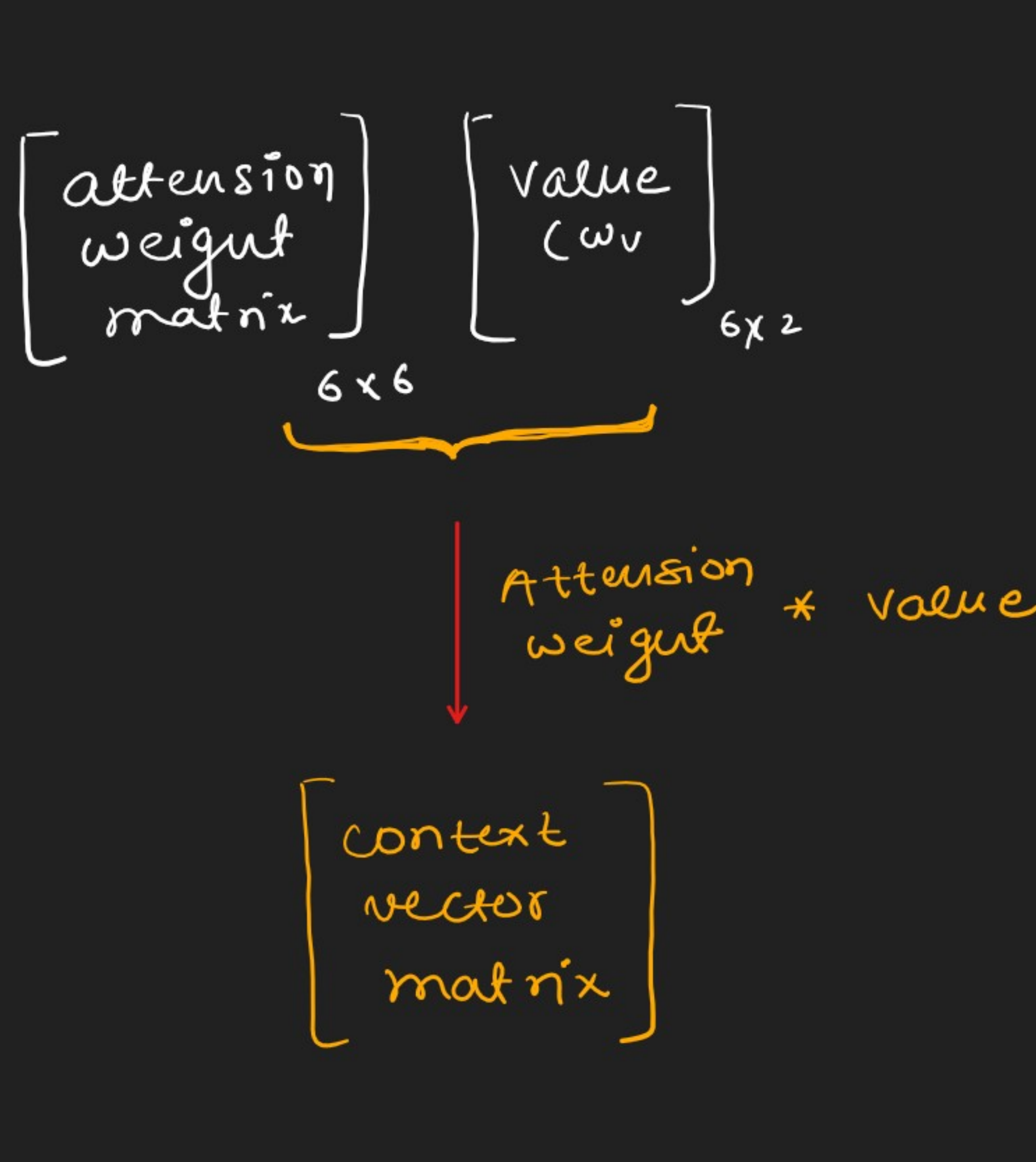
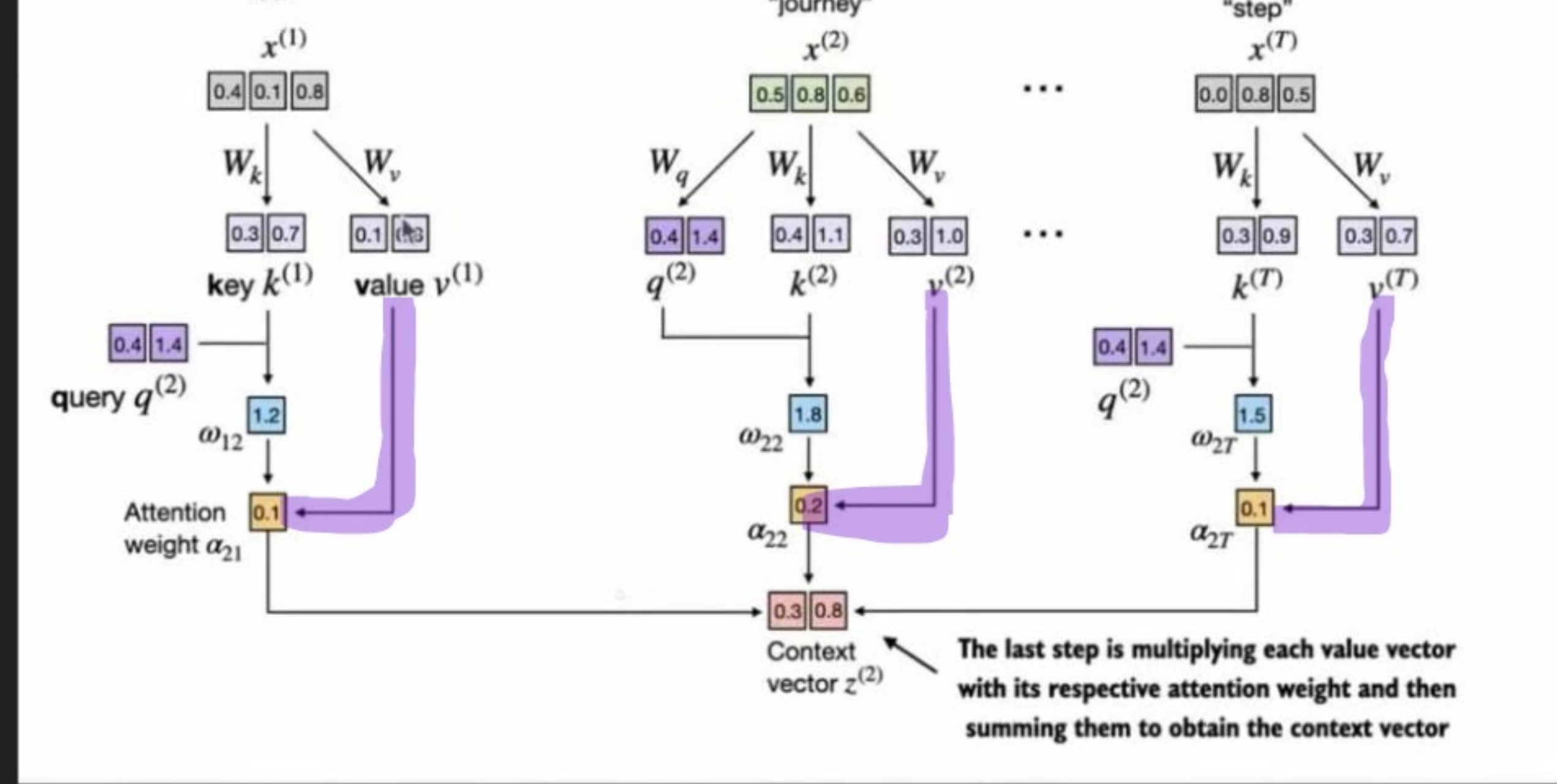
Then we compute the attention weight

③ Attention weight



④ context vector

Final steps compute context vector



⑤

why do we use the term: key, Query & value?

Query: Analogous to search Query in a database.

It represent the current token the model focus on.

key: In attention mechanism, each item in input sequence has a key, key are used to match with the Query.

value: It represent the actual content or representation of the input items. once the model determine which key (which parts of the input) are most relevant to the Query (current) focus items, it retrieves the corresponding values

⑥ Implementing a compact self-attention python class

These 3 matrix are the trainable weight matrix

